Guide to Exporting MoE Custom ONNX Nodes and Converting Them to RKNN

Release Version: V1.2.0

Release Date: 2026-10-29

Revision History

Version	Date	Change Description
V1.2.0	2026-10-29	Initial version: MoE custom ONNX node export and RKNN conversion guide.

This document explains how to export the routed-expert portion of an MoE model (router, top-k selection, and expert FFNs) as `rknn::exUnifiedMoE`, then convert it using `load_onnx` or `load_llm` in RKNN3-Toolkit.

This guide applies to RKNN3-Toolkit version 1.2.0 and later.

1. Conversion Boundaries and Validation Order

```
HF/PyTorch MoE
  ├─ routed experts → ONNX rknn::exUnifiedMoE ─┐
  └─ shared expert  → standard ONNX operators ─┴→ load_onnx/load_llm → build →
export_rknn
```

`exUnifiedMoE` outputs only the routed-expert result, with the same shape and data type as `hidden_states`. The shared expert and the operation that combines its output with the routed-expert result must remain outside the node and follow the original model's computation; otherwise, the result will omit the shared expert's contribution.

First check the inputs and attributes of `exUnifiedMoE`. Then extract and convert a single MoE node and compare its numerical accuracy. Finally, convert the full model and validate it on the target device.

2. ONNX Definition of `rknn::exUnifiedMoE`

Use `domain="rknn"` and declare `rknn` in the model's `opset_import` (the examples use version `1`) so that the model passes `onnx.checker`. The Toolkit identifies the operator by the node's `op_type=exUnifiedMoE`; the domain version in the examples is not a fixed requirement of the operator.

2.1 Operator Inputs and Output

`exUnifiedMoE` has one output. Inputs must be supplied **consecutively** in the order defined below: omit optional inputs that are not enabled, without using empty strings as placeholders. Let `B=batch size`, `N=number of tokens`, `H=hidden_size`, `E=n_routed_experts`, `K=num_experts_per_token`, and `I=expert intermediate dimension`. Currently, only `B=1` is supported.

Order	Input	ONNX shape	Data type	When to supply
1	`hidden_states`	`[1,N,H]`	float32	Required
2	`token_id_fix_experts`	`[B,N,K]`	int32	Supply fixed expert IDs when `token_id_fix_experts=1`; supported only in simulation
3	`norm_before_router_scales`	`[H]`	float32	Supply when `norm_before_router_with_scales=1`
4	`router_weights`	`[H,E]`	float32	Required
5	`router_bias`	`[E]`	float32	Supply when `router_bias=1`
6	`norm_before_ffn_scales`	`[H]`	float32	Supply when `norm_before_ffn_with_scales=1`
7	`score_select_bias`	`[E]`	float32	Supply when `score_select_bias=1`; must be constant at conversion time
8	`per_expert_scale`	`[E]`	float32	Supply when `per_expert_scale=1`; must be constant at conversion time
Following inputs	Expert weights	See the table below	float32	Supply one packed set when `expert_weight_packed='batch'`; otherwise, supply E sets of expert inputs in expert-index order

The input order and shapes within each expert weight set are:

`gate_up_packed`	Input order within each set (supply biases only when enabled)	Shapes without batch packing; in `batch` mode, prepend an E dimension to each shape
`0`	`gate_w`, `gate_b?`, `up_w`, `up_b?`, `down_w`, `down_b?`	`[H,I]`, `[I]`, `[H,I]`, `[I]`, `[I,H]`, `[H]`

gate_up_packed	Input order within each set (supply biases only when enabled)	Shapes without batch packing; in batch mode, prepend an E dimension to each shape
1	gate_up_w, gate_up_b?, down_w, down_b?	[H,2I]; [2I]; [I,H]; [H]

batch packing requires all expert weights to have the same I, for example, gate_up_w=[E,H,2I] and down_w=[E,I,H]. All matrix weights use the [in,out] layout for the mathematical operation x @ w. PyTorch nn.Linear.weight usually has the [out,in] layout and must be transposed in the **user's export script**. Do not transpose weights that already use [in,out]. Biases are not transposed.

When gate_up_cross_packed=0, the first half of the last dimension contains gate values and the second half contains up values. When it is 1, even indices contain gate values and odd indices contain up values. Interleaved packing requires gate_up_packed=1, and gate/up biases must either both be enabled or both be disabled.

For example, the inputs may be:

```
hidden_states
...
per_expert_scale
gate_w0
up_w0
down_w0
gate_w1
up_w1
down_w1
...
gate_we
up_we
down_we
```

2.2 Operator Attributes

The following are **attribute names on the exUnifiedMoE node**. When using the PyTorch legacy symbolic method, append a type suffix to each attribute name in graph.op, for example, n_routed_experts_i=E; the exported attribute name does not include _i. Use 0/1 for integer flags. Omit attributes that are not enabled instead of setting them to the string "None".

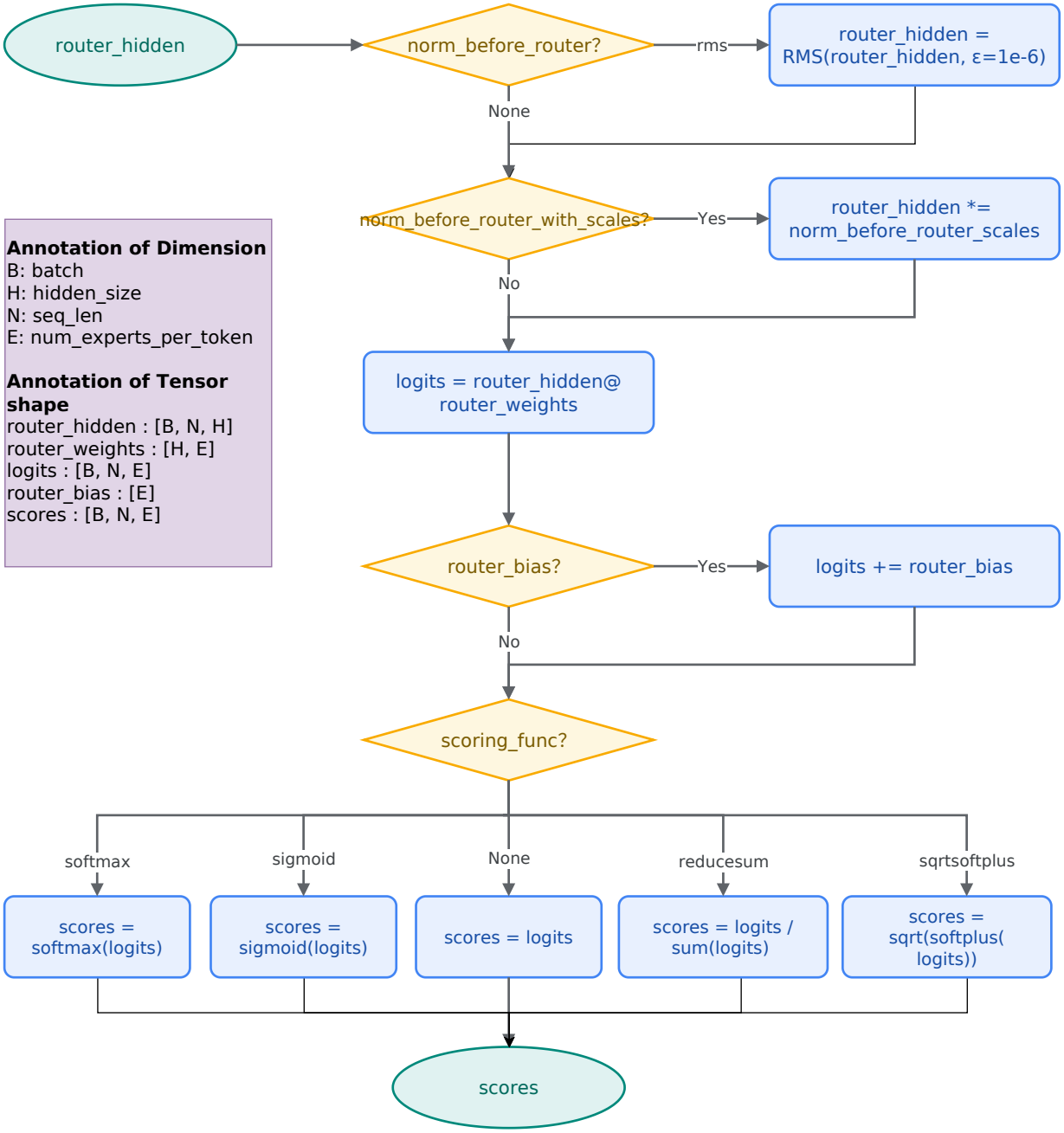
Attribute	Values and behavior
n_routed_experts	Required integer; the number of routed experts, abbreviated as E, excluding the shared expert. Currently, E must be <= 2048
num_experts_per_token	Required integer; the number of experts selected per token, abbreviated as K. Must satisfy 1 ≤ K ≤ E and K <= 16
topk_method	greedy (default) or group_limited_greedy; the latter also requires n_group and topk_group
n_group	Number of groups when using group_limited_greedy; must be a positive integer with E % n_group == 0

Attribute	Values and behavior
`topk_group`	Number of selected groups when using `group_limited_greedy`; must satisfy `1 ≤ topk_group ≤ n_group` and `K ≤ topk_group * E / n_group`
`scoring_func`	`softmax`, `sigmoid`, `reducesum`, or `sqrtsoftplus`; omit to use router logits directly. `reducesum` normalizes each row by its sum
`score_select_bias`	Defaults to `0`; when set to `1`, supply the input with the same name. The bias affects only the expert IDs selected by top-k; expert weighting scores are still taken from the scores before adding the bias
`norm_selected_score`	`softmax`, `sigmoid`, `reducesum`, or `sigmoid_reducesum`, or omit; applies to the selected K scores
`routed_scaling_factor`	Scaling factor for the selected expert scores; defaults to `1.0`
`norm_before_router`, `norm_before_ffn`	Can be set to `rms`, applied to the router input and expert FFN input, respectively; no normalization by default
`norm_before_router_with_scales`, `norm_before_ffn_with_scales`	Default to `0`; when set to `1`, enable the corresponding `rms` normalization and supply a `[H]` scale
`router_bias`, `gate_bias`, `up_bias`, `down_bias`	Default to `0`; when set to `1`, supply the corresponding bias
`expert_weight_packed`	`batch` indicates one set of weights stacked along an E dimension; omit to supply E separate weight sets
`gate_up_packed`, `gate_up_cross_packed`	Default to `0`; indicate combined gate/up weights and interleaved packing of the combined weights and biases, respectively
`activation`	`silu` / `swish`, `gelu` / `gelu_erf`, or `gelu_tanh`; defaults to `silu`
`activation_alpha`	The `alpha` in `x * sigmoid(alpha * x)` for `silu` / `swish`; defaults to `1.0`
`gate_clip_mode`, `up_clip_mode`	Control clipping of the gate/up projection results, respectively. `NONE` (default) disables clipping, `MAX` applies an upper bound only, and `MIN_MAX` applies lower and upper bounds. Clipping occurs before multiplying gate and up
`gate_clip_min`, `up_clip_min`	Lower clipping bounds for gate/up; used only when the corresponding mode is `MIN_MAX`. Default to `0.0`
`gate_clip_max`, `up_clip_max`	Upper clipping bounds for gate/up; used when the corresponding mode is `MAX` or `MIN_MAX`. Default to `0.0`
`up_offset`	Offset added to up after clipping; defaults to `0.0`
`per_expert_scale`	Defaults to `0`; when set to `1`, supply the input with the same name
`routed_scores_before_fn`	Defaults to `0`, which multiplies the down output by the routing score; when set to `1`, multiplies the expert FFN input by the routing score
`token_id_fix_experts`	Defaults to `0`; when set to `1`, supply fixed expert IDs. Supported only in simulation

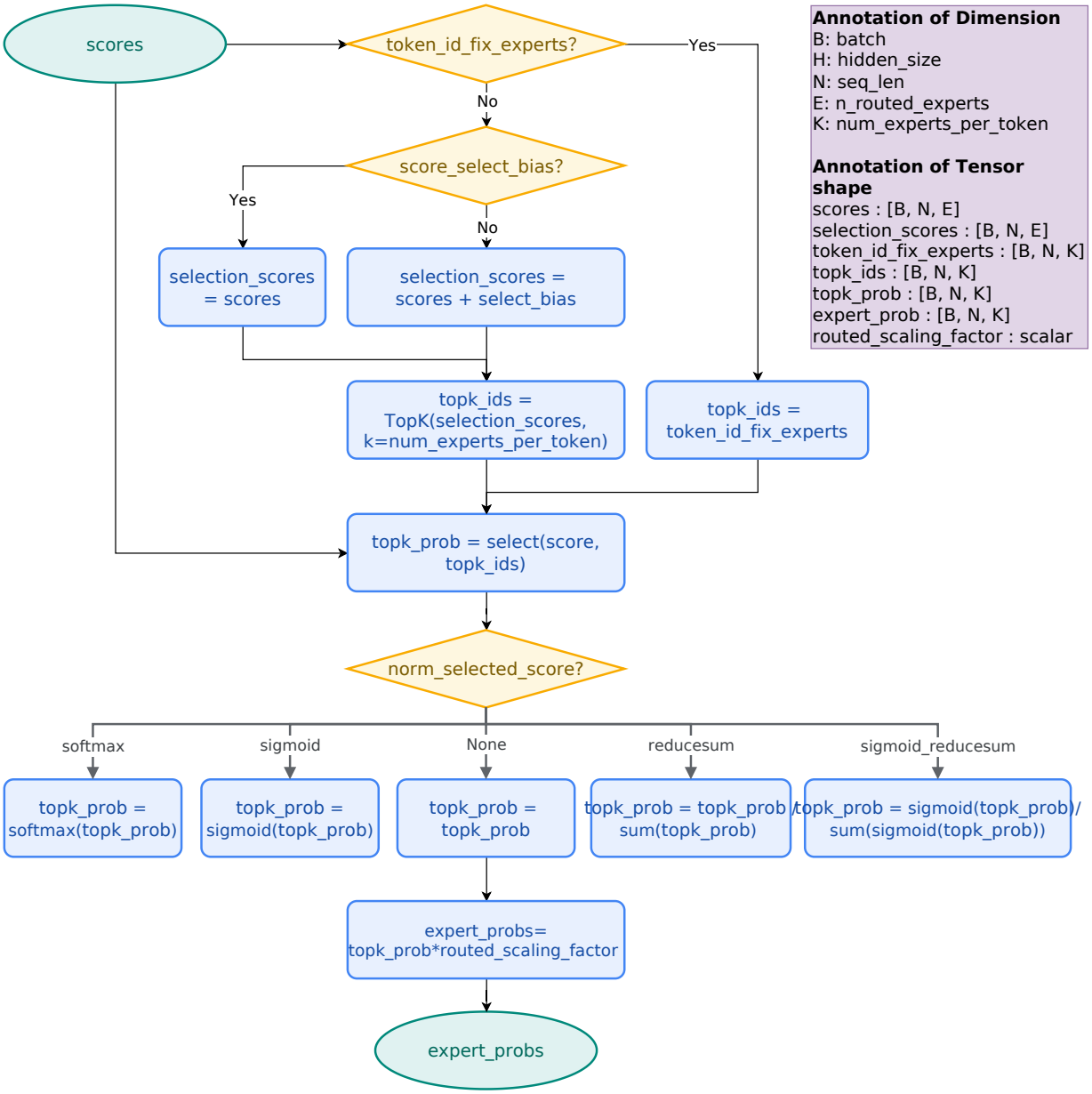
Capacity-based routing (`expert_capacity` , `expert_policy`), fixed expert ID routing (`token_id_fix_experts`), and `topk_method=group_limited_greedy` are not yet fully supported and will raise errors when generating an RKNN model.

Fixed expert ID routing is for simulation only: look up expert IDs by token ID outside the node and supply them with shape `[B,N,K]` . Each ID must be in `[0,E)` . The node still computes router scores; the fixed IDs replace only top-k selection. The selected scores are still processed using `norm_selected_score` and `routed_scaling_factor` . If the original model also outputs router logits, preserve that path outside the node.

2.3 Score Computation Flow



2.4 Top-k Selection and Expert Weighting Flow



2.5 A Minimal ONNX Model You Can Generate Directly

The following script depends only on `numpy` and `onnx` and checks the basic input order, weight layout, and attributes. It generates a **structural example** with random weights and cannot be used to validate the accuracy of a real model. Save it as `make_minimal_moe.py` and run `python make_minimal_moe.py`.

```
import numpy as np
import onnx
from onnx import TensorProto, helper, numpy_helper

H, I, E, K, N = 8, 16, 4, 2, 3
rng = np.random.default_rng(0)

def weight(name, shape):
    array = rng.standard_normal(shape).astype(np.float32) * 0.01
    return numpy_helper.from_array(array, name=name)

# Batch packing: hidden, router, gate_up, down; omit inputs for disabled
biases/scales.
initializers = [
    weight("router_w", (H, E)),
    weight("gate_up_w", (E, H, 2 * I)),
    weight("down_w", (E, I, H)),
]
node = helper.make_node(
    "exUnifiedMoE", ["hidden", "router_w", "gate_up_w", "down_w"],
    ["routed_out"], name="moe_0", domain="rknn",
    n_routed_experts=E, num_experts_per_token=K,
    topk_method="greedy", scoring_func="softmax",
    norm_selected_score="reducesum", expert_weight_packed="batch",
    gate_up_packed=1, activation="silu",
)
graph = helper.make_graph(
    [node], "minimal_moe",
    [helper.make_tensor_value_info("hidden", TensorProto.FLOAT, [1, N, H])],
    [helper.make_tensor_value_info("routed_out", TensorProto.FLOAT, [1, N, H])],
    initializer=initializers,
)
model = helper.make_model(
    graph, opset_imports=[helper.make_operatorsetid("", 19),
                          helper.make_operatorsetid("rknn", 1)],
)
onnx.checker.check_model(model)
onnx.save_model(model, "minimal_moe.onnx")
```

2.6 Exporting a Real Model from HF/PyTorch

In the model export script, use `torch.autograd.Function` to wrap the routed-expert portion: `forward` retains computation equivalent to the original model for numerical comparison before export, while `symbolic` generates `rknn::exUnifiedMoE`. Before calling `Function.apply`, arrange weights in the `[in,out]` layout as described in Section 2. Do not call the PyTorch Tensor method `.t()` on an ONNX `Value` inside `symbolic`. For example:

```
@staticmethod
def symbolic(g, hidden, router_w, gate_up_w, down_w):
    out = g.op(
        "rknn::exUnifiedMoE", hidden, router_w, gate_up_w, down_w,
        n_routed_experts_i=E, num_experts_per_token_i=K,
        scoring_func_s="softmax", norm_selected_score_s="reducesum",
        expert_weight_packed_s="batch", gate_up_packed_i=1,
        activation_s="silu", outputs=1,
    )
    out.setType(hidden.type())
    return out
```

Obtain `E` and `K` from the model configuration; implement `forward` and the model replacement logic for the specific model. If the original weights are `gate_up_proj.weight=[E,2I,H]` and `down_proj.weight=[E,H,I]`, use `transpose(1, 2).contiguous()` to convert them to `[E,H,2I]` and `[E,I,H]`, respectively. For the router's `Linear.weight=[E,H]`, use `transpose(0, 1).contiguous()` to obtain `[H,E]`. Pass weights directly if they already use the `[in,out]` layout. Check the original model's routing scores, normalization, biases, and shared expert individually; do not choose attributes based only on the model name.

Use a PyTorch legacy exporter that supports `symbolic`, for example:

```
torch.onnx.export(
    model, inputs, "model.onnx", opset_version=19,
    export_params=True, external_data=True,
    custom_opsets={"rknn": 1}, dynamo=False,
)
```

Export parameters may differ across PyTorch versions. Check that the **actual exported ONNX model** contains `rknn::exUnifiedMoE`. If the model uses external data, retain all weight files referenced by the ONNX model during conversion.

If the original model's gate/up weights are interleaved, set `gate_up_cross_packed=1`. Export activation, clipping, offsets, and biases according to the original model configuration as well. Mask logic outside the routed-expert computation must remain outside the node.

3. Checks and Troubleshooting Without Toolkit Source Code

3.1 Converting a Single MoE Node

Use `load_onnx(inputs=..., outputs=...)` to extract a subgraph. First find `input[0]` and `output[0]` of the desired MoE node. To validate the complete module, including the shared expert, select the tensor after the outputs are combined as the output. Set `input_size_list` to the actual `[1,N,H]` shape of the input:

```
import onnx
from rknn.api import RKNN

path = "model.onnx"
model = onnx.load(path, load_external_data=False)
moe = next(n for n in model.graph.node if n.op_type == "exUnifiedMoE")
print("cut:", moe.name, moe.input[0], "->", moe.output[0])

N, H = 3, 8  # Replace with the actual token count and hidden_size of this MoE
layer.
rknn = RKNN(verbose=True)
try:
    assert rknn.config(target_platform="rk1820", core_num=8,
                       enable_moe_tc=False) == 0
    assert rknn.load_onnx(model=path, inputs=[moe.input[0]],
                          input_size_list=[[1, N, H]],
                          outputs=[moe.output[0]]) == 0
    assert rknn.build(do_quantization=False) == 0
    assert rknn.export_rknn("single_moe.rknn") == 0
finally:
    rknn.release()
```

Replace `rk1820` / `core_num` with the actual target configuration. When validating only the routing node, use the original model's routed-expert output as the reference. With the same inputs and precision settings, compare the shape, maximum absolute error, and cosine similarity. After the single-layer check passes, validate the full model and the target device. The example initially disables quantization; build quantized models with the actual configuration and compare them separately.

If the full model meets the LLM import requirements, use `RKNN.config(..., llm_config=...)`, `load_llm("model.onnx")`, `build(...)`, and `export_rknn(...)`. Check the return code at every step and save the complete logs.

3.2 Valid Token Count for Fixed Shapes

This feature has not yet been implemented and is currently unsupported!

`enable_moe_tc` defaults to `False`. To specify the valid token count for fixed-shape prefill, set it to `True` after confirming that the target environment supports it. Once successfully enabled, the exported model has an additional `[1] int32` input (usually named `MoE_Tc`). During inference, supply the valid token count according to the actual input list; its value must be in `[0,N]`.

Enabling this feature requires all MoE nodes in the model to have the same sequence length, which must be a known positive integer. `MoE_Tc` does not need to be added to the original ONNX `exUnifiedMoE` input list. Keep the default setting for the initial numerical comparison.

3.3 Common Symptoms and Remedies

Symptom/log	First checks
No `exUnifiedMoE` node in the ONNX model	Check whether the exporter calls `symbolic` and whether the target MoE layers have been replaced
Missing required attributes, or `hidden_states must be [1,N,H]`	Check `E`, `K`, and the input's batch size/static shape
Incorrect number of inputs	Check optional input flags and the number of expert sets against Section 2; do not supply empty placeholders
Incorrect expert weight shapes	Check `Linear` weight transposes, the leading E dimension in `batch` mode, `2I` for gate/up, and `[I,H]` for down
Unsupported attribute combination; build fails	Check routing methods and attribute values against Section 2.1, especially capacity-based routing and fixed expert ID routing
`per_expert_scale` is not constant	Export the `[E]` scale as an ONNX initializer/constant
`score_select_bias` is not an FP32 constant	Export the `[E]` bias as an FP32 initializer/constant
`MoE_Tc` cannot be enabled	Check whether all MoE nodes have the same sequence length; disable `enable_moe_tc` for the initial numerical comparison
Build succeeds, but results differ significantly	First compare unquantized single-layer results, then check score normalization, gate/up layout, activation and clipping, the shared expert, and external masks
Failure occurs only on the target device	Check the Toolkit and device runtime versions; retain input samples, the model, and logs for troubleshooting

When reporting an issue, provide the Toolkit and device runtime versions, target chip, a reproducible ONNX model and its external data, node attributes and input shapes, conversion logs, and outputs from both the original model and RKNN using the same inputs. If the weights cannot be shared, use a model with the same structure and random weights to reproduce export issues; accuracy issues still require numerical comparison using the real model.